BINARY SEARCH TREE

Java • Fundamentals to Interview Mastery

Complete BST handbook

Search • Insert • Delete • Min/Max • Validate • Kth Smallest/Largest
Successor/Predecessor • LCA • Sorted Array to BST • Recover BST

Designed for learning the reasoning, tracing the pointer changes, writing clean Java, and explaining complexity in interviews.

Read the explanation → trace the example → code it yourself → solve the practice set.

How to use this book

This handbook uses one consistent Java node definition and assumes a BST with unique keys unless a section explicitly says otherwise. A BST follows the ordering rule: every key in the left subtree is smaller than the node, and every key in the right subtree is greater.

• **First pass:** understand the ordering invariant and draw each operation by hand.

• **Second pass:** type the code without copying, then test empty tree, one node, root changes, and missing keys.

• **Interview pass:** explain why each branch is safe and state worst-case complexity.

• **Important:** ordinary BST operations are O(h), not always O(log n). They become O(log n) only when height stays logarithmic (for example, in a balanced BST).

Core invariant: for every node x, all values in x.left are less than x.val and all values in x.right are greater than x.val. This must hold for the entire subtrees, not just immediate children.

Contents

• 1. BST fundamentals, terminology, shape and complexity

• 2. Java Node class, construction and reusable utilities

• 3. Search and Insert (recursive + iterative)

• 4. Minimum and Maximum

• 5. Delete: all three structural cases

• 6. Validate BST: bounds and inorder approaches

• 7. Kth Smallest and Kth Largest

• 8. Inorder Successor and Predecessor

• 9. Lowest Common Ancestor in BST

• 10. Sorted Array to Height-Balanced BST

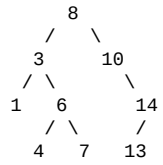
• 11. Recover BST with two swapped nodes

• 12. Testing, edge cases, interview traps and problem set

1. BST fundamentals

What makes a tree a BST?

A Binary Search Tree is a binary tree with an ordering rule. Each node has at most two children. For a node with key k, every key in its left subtree must be < k and every key in its right subtree must be > k. This strict version assumes unique keys.



For node 8, the entire left subtree contains 1, 3, 4, 6, 7 (all < 8); the entire right subtree contains 10, 13, 14 (all > 8).

Why BST search is efficient

At each node, compare the target with the current key. If target is smaller, the answer can only be in the left subtree; if larger, only in the right subtree. One whole side is discarded at each step.

Height and shape

Shape	Height (edges)	Typical operation cost
Balanced	O(log n)	O(log n)
Completely skewed	O(n)	O(n)
Empty tree	-1 by edge-height convention	No node visited
Single node	0 by edge-height convention	O(1)

A BST can become skewed if values arrive in sorted order. Inserting 1, 2, 3, 4, 5 creates a chain. A self-balancing tree such as AVL or Red-Black Tree controls height; a plain BST does not.

Inorder traversal is sorted

Inorder means Left → Node → Right. In a valid BST with unique keys, inorder traversal visits keys in strictly increasing order. This fact powers validation, kth smallest, successor, and recovery.

```
Inorder of sample tree:
1, 3, 4, 6, 7, 8, 10, 13, 14
```

Interview sentence: “BST operations take O(h), where h is the tree height. In a balanced tree h = O(log n); in a skewed tree h = O(n).”

2. Java foundation: Node and utilities

Use this node definition throughout the book. The value is an int and left/right point to child nodes.

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }
}
```

Build a sample BST

```
class BST {
    TreeNode root;

    void insert(int value) {
        root = insert(root, value);
    }

    private TreeNode insert(TreeNode node, int value) {
        if (node == null) return new TreeNode(value);

        if (value < node.val) {
            node.left = insert(node.left, value);
        } else if (value > node.val) {
            node.right = insert(node.right, value);
        }
        // Duplicate values are ignored in this implementation.
        return node;
    }
}
```

The assignment `root = insert(root, value)` matters: inserting into an empty tree must update root. Recursive calls also reassign `node.left` or `node.right` because a new child may be created.

Iterative inorder utility

```
import java.util.*;

static List<Integer> inorder(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        curr = stack.pop();
        result.add(curr.val);
        curr = curr.right;
    }
    return result;
}
```

Time: O(n), because every node is pushed and popped once. Auxiliary space: O(h) for the stack, worst-case O(n).

A compact test driver

```
public static void main(String[] args) {
    BST bst = new BST();
    int[] values = {8, 3, 10, 1, 6, 14, 4, 7, 13};

    for (int x : values) bst.insert(x);

    System.out.println(inorder(bst.root));
    // [1, 3, 4, 6, 7, 8, 10, 13, 14]
}
```

3. Search in a BST

Logic

• If current node is null, the key does not exist.

• If target equals current value, return the current node.

• If target is smaller, search left; otherwise search right.

• Never search both subtrees: the BST invariant rules one side out.

Recursive search

```
static TreeNode searchRecursive(TreeNode root, int key) {
    if (root == null || root.val == key) {
        return root;
    }

    if (key < root.val) {
        return searchRecursive(root.left, key);
    }
    return searchRecursive(root.right, key);
}
```

Iterative search (no call-stack usage)

```
static TreeNode searchIterative(TreeNode root, int key) {
    TreeNode curr = root;

    while (curr != null) {
        if (key == curr.val) return curr;

        if (key < curr.val) {
            curr = curr.left;
        } else {
            curr = curr.right;
        }
    }
    return null;
}
```

Dry run: search 7

Step	Current	Comparison	Next
1	8	7 < 8	left → 3
2	3	7 > 3	right → 6
3	6	7 > 6	right → 7
4	7	equal	found

Time: O(h). Recursive auxiliary space: O(h); iterative auxiliary space: O(1). If the tree is balanced, O(log n); if skewed, O(n).

Common bug: returning `root` after the loop even when the key was not found. The correct result is `null`.

4. Insert into a BST

Rule

Compare the new value as you descend. Smaller goes left, larger goes right. When you reach a null child position, create the node there. This implementation ignores duplicates.

Recursive insert

```
static TreeNode insertRecursive(TreeNode root, int value) {
    if (root == null) {
        return new TreeNode(value);
    }

    if (value < root.val) {
        root.left = insertRecursive(root.left, value);
    } else if (value > root.val) {
        root.right = insertRecursive(root.right, value);
    }
    return root;
}
```

Iterative insert

```
static TreeNode insertIterative(TreeNode root, int value) {
    if (root == null) return new TreeNode(value);

    TreeNode curr = root;
    TreeNode parent = null;

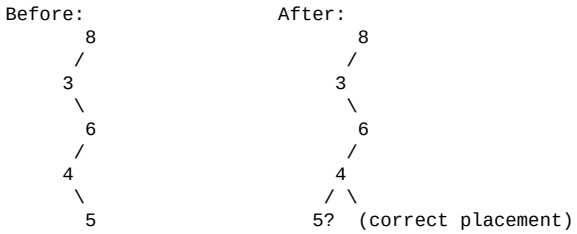
    while (curr != null) {
        parent = curr;

        if (value < curr.val) {
            curr = curr.left;
        } else if (value > curr.val) {
            curr = curr.right;
        } else {
            return root; // duplicate: ignore
        }
    }

    if (value < parent.val) {
        parent.left = new TreeNode(value);
    } else {
        parent.right = new TreeNode(value);
    }
    return root;
}
```

Dry run: insert 5

Start at 8: 5 < 8, go left to 3. At 3: 5 > 3, go right to 6. At 6: 5 < 6, go left to 4. At 4: 5 > 4, its right child is null; attach 5 there.



The actual final local shape is 6.left = 4 and 4.right = 5. Inorder remains sorted.

Time: O(h). Recursive space: O(h); iterative space: O(1).

Root reminder: always keep the returned root: `root = insertRecursive(root, value);`.

5. Minimum and Maximum

The minimum key is the leftmost node. The maximum key is the rightmost node. This follows directly from the BST ordering rule.

Find minimum and maximum nodes

```
static TreeNode findMinNode(TreeNode root) {
    if (root == null) return null;

    TreeNode curr = root;
    while (curr.left != null) {
        curr = curr.left;
    }
    return curr;
}

static TreeNode findMaxNode(TreeNode root) {
    if (root == null) return null;

    TreeNode curr = root;
    while (curr.right != null) {
        curr = curr.right;
    }
    return curr;
}
```

Return values instead of nodes

```
static Integer minValue(TreeNode root) {
    TreeNode node = findMinNode(root);
    return node == null ? null : node.val;
}

static Integer maxValue(TreeNode root) {
    TreeNode node = findMaxNode(root);
    return node == null ? null : node.val;
}
```

Why `Integer`? An empty tree has no minimum or maximum. Returning `null` makes that case explicit; alternatively, throw an exception or return `OptionalInt`.

Time: O(h). Auxiliary space: O(1).

Connection to delete: the inorder successor of a node with two children is the minimum node in its right subtree.

6. Delete from a BST — the most important operation

Deletion is harder because removing a node can break the structure. Search for the key first, then handle one of three cases.

Case 1: node is a leaf

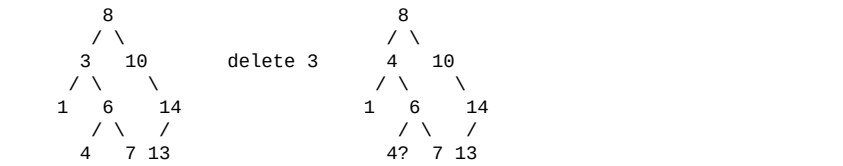
It has no children. Replace the node with null. Example: deleting 1 from the sample tree simply sets its parent's left link to null.

Case 2: node has exactly one child

Replace the node with its only child. If node.left is null, return node.right; otherwise, if node.right is null, return node.left. The child is linked directly to the deleted node's parent.

Case 3: node has two children

Find the inorder successor: the smallest node in the right subtree. Copy its value into the node being deleted, then delete the successor from the right subtree. The successor has no left child, so its own deletion reduces to Case 1 or Case 2.




```
For deleting 3 (two children):
successor = minimum of right subtree = 4
copy 4 into node 3, then remove original 4.
Final left subtree root is 4, with children 1 and 6; 6.left becomes null.
```

Complete recursive delete

```
static TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val) {
        root.left = deleteNode(root.left, key);
    } else if (key > root.val) {
        root.right = deleteNode(root.right, key);
    } else {
        // Found the node.

        // Case 1 and Case 2:
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Case 3: two children.
        TreeNode successor = findMinNode(root.right);
        root.val = successor.val;
        root.right = deleteNode(root.right, successor.val);
    }
    return root;
}
```

Why does this work?

• The recursive search follows exactly one valid BST path.

• Returning the only child reconnects the parent to the remaining subtree.

• The successor is greater than every value in the deleted node's left subtree.

• The successor is the smallest value in the right subtree, so replacing the node preserves ordering.

• Deleting the copied successor removes the duplicate value from its old position.

Time: O(h) for search plus successor discovery and removal; still O(h) overall. Recursive space: O(h).

6A. Delete: trace each case

Leaf deletion

Tree: 8 with left child 3; 3 has leaf child 1. Delete 1. Search path is 8 → 3 → 1. Since 1 has no children, return null to the parent; then 3.left becomes null.

One-child deletion

Suppose node 10 has only right child 14. Delete 10: return 14. Its parent now points directly to 14. If the deleted node was root, the returned child becomes the new root.

Two-child deletion

Delete 8 from the sample. Its successor is the minimum of its right subtree: 10. Copy 10 into the root, then delete the original 10 from the right subtree. If that 10 had right child 14, that child is reconnected.

Iterative deletion (alternative)

A clean recursive version is usually easiest to explain. For an iterative version, first locate `curr` and `parent`, then choose a replacement subtree. If the node has two children, locate the inorder successor in the right subtree, copy its value, and remove that successor (which has at most one child).

```
static TreeNode deleteIterative(TreeNode root, int key) {
    TreeNode parent = null;
    TreeNode curr = root;

    // 1) Search for target.
    while (curr != null && curr.val != key) {
        parent = curr;
        curr = (key < curr.val) ? curr.left : curr.right;
    }
    if (curr == null) return root; // key absent

    // 2) For two children, copy successor value into target.
    if (curr.left != null && curr.right != null) {
        TreeNode succParent = curr;
        TreeNode succ = curr.right;

        while (succ.left != null) {
            succParent = succ;
            succ = succ.left;
        }

        curr.val = succ.val;
        parent = succParent;
        curr = succ; // now delete successor node
    }

    // 3) curr has at most one child.
    TreeNode child = (curr.left != null) ? curr.left : curr.right;

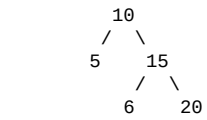
    if (parent == null) {
        return child; // deleting root
    } else if (parent.left == curr) {
        parent.left = child;
    } else {
        parent.right = child;
    }
    return root;
}
```

This iterative version is O(h) time and O(1) auxiliary space. The identity check `parent.left == curr` is safe because it compares node references, not values.

Interview trap: forgetting to assign the returned root after deleting the root. Use `root = deleteNode(root, key);`.

7. Validate a BST

A common wrong approach checks only `node.left.val < node.val` and `node.right.val > node.val`. That is insufficient: a deeper descendant can violate an ancestor's constraint.




```
This is NOT a BST because 6 is in the right subtree of 10,
even though 6 < 15. Every node in 10.right must be > 10.
```

Method 1: lower and upper bounds (recommended)

Carry the allowed open interval down the tree. A node must be strictly greater than the lower bound and strictly smaller than the upper bound. Moving left changes the upper bound to the current value; moving right changes the lower bound.

```
static boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

static boolean validate(TreeNode node, long low, long high) {
    if (node == null) return true;

    if (node.val <= low || node.val >= high) {
        return false;
    }

    return validate(node.left, low, node.val)
        && validate(node.right, node.val, high);
}
```

Why `long` bounds? Because a valid tree may contain Integer.MIN_VALUE or Integer.MAX_VALUE. Using int sentinels can accidentally reject those boundary values.

Method 2: inorder must be strictly increasing

```
static boolean isValidBSTInorder(TreeNode root) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    long previous = Long.MIN_VALUE;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        curr = stack.pop();

        if (curr.val <= previous) return false;
        previous = curr.val;
        curr = curr.right;
    }
    return true;
}
```

Both methods take O(n) time. Bounds recursion uses O(h) call stack; iterative inorder uses O(h) stack.

Duplicates: these implementations reject duplicates. If an interviewer defines duplicates as always-left or always-right, adjust the inequalities consistently. Never mix policies.

8. Kth Smallest and Kth Largest

Inorder traversal of a BST gives ascending order. Therefore the kth smallest is the kth node visited in inorder. Reverse inorder (Right → Node → Left) gives descending order; the kth visited node is kth largest.

Kth smallest — iterative inorder

```
static Integer kthSmallest(TreeNode root, int k) {
    if (k <= 0) return null;

    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        curr = stack.pop();
        k--;

        if (k == 0) return curr.val;
        curr = curr.right;
    }
    return null; // k > number of nodes
}
```

Kth largest — reverse inorder

```
static Integer kthLargest(TreeNode root, int k) {
    if (k <= 0) return null;

    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.right;
        }

        curr = stack.pop();
        k--;

        if (k == 0) return curr.val;
        curr = curr.left;
    }
    return null;
}
```

Example

For inorder [1, 3, 4, 6, 7, 8, 10, 13, 14], k=4 gives 6 as the 4th smallest. The 4th largest is 10 (descending order: 14, 13, 10, 8, ...).

Time: O(h + k) in a typical traversal; worst-case O(n). Auxiliary space: O(h). This stops early and does not need to visit every node.

Edge cases: k = 0 or negative, empty tree, k greater than node count. Decide whether the API returns null, OptionalInt, or throws an exception.

8A. Optimized repeated kth queries: subtree sizes

If many kth queries are asked on a tree that does not change, store the size of each subtree. At node x, let L = size(x.left). If k = L + 1, x is the answer; if k ≤ L, go left; otherwise go right with k - L - 1.

```
class SizeNode {
    int val, size = 1;
    SizeNode left, right;

    SizeNode(int val) { this.val = val; }
}

static int size(SizeNode node) {
    return node == null ? 0 : node.size;
}

static SizeNode kthBySize(SizeNode root, int k) {
    if (root == null || k < 1 || k > root.size) return null;

    int leftSize = size(root.left);

    if (k == leftSize + 1) return root;
    if (k <= leftSize) return kthBySize(root.left, k);

    return kthBySize(root.right, k - leftSize - 1);
}
```

With correct size metadata, query time is O(h). But every insert/delete must update sizes along the affected path. If metadata is stale, the answer is wrong.

Complexity comparison

Approach	One query	Extra data / cost
Inorder stack	O(h + k)	No metadata; O(h) stack
Subtree sizes	O(h)	Maintain size on updates
Store full sorted list	O(1) after preprocessing	O(n) build + O(n) memory; updates costly

9. Inorder Successor and Predecessor

Successor: the smallest key strictly greater than x. **Predecessor:** the largest key strictly smaller than x. These definitions also work when x is not present, as “next greater” and “next smaller”.

Successor when the key exists

• If node has a right subtree, successor is the minimum node in that right subtree.

• Otherwise, move upward: the successor is the lowest ancestor for which the current node lies in that ancestor's left subtree.

When nodes do not have parent pointers, the simplest BST method is to search from root while remembering the best candidate.

```
static TreeNode successor(TreeNode root, int key) {
    TreeNode candidate = null;
    TreeNode curr = root;

    while (curr != null) {
        if (key < curr.val) {
            candidate = curr;       // curr may be the next greater
            curr = curr.left;
        } else if (key > curr.val) {
            curr = curr.right;
        } else {
            if (curr.right != null) return findMinNode(curr.right);
            return candidate;
        }
    }
    return candidate; // also gives next greater if key is absent
}
```

Predecessor

```
static TreeNode predecessor(TreeNode root, int key) {
    TreeNode candidate = null;
    TreeNode curr = root;

    while (curr != null) {
        if (key > curr.val) {
            candidate = curr;       // curr may be the next smaller
            curr = curr.right;
        } else if (key < curr.val) {
            curr = curr.left;
        } else {
            if (curr.left != null) return findMaxNode(curr.left);
            return candidate;
        }
    }
    return candidate; // also gives next smaller if key is absent
}
```

Time: O(h), auxiliary space: O(1). If the key is present and the relevant child subtree exists, min/max search remains within O(h).

Example

For keys [1, 3, 4, 6, 7, 8, 10, 13, 14], successor(7)=8 and predecessor(7)=6. successor(14) does not exist; predecessor(1) does not exist.

Subtle point: do not confuse successor with “right child”. The successor may be an ancestor. Likewise, predecessor may be an ancestor.

10. Lowest Common Ancestor (LCA) in a BST

The LCA of p and q is the lowest node that has both as descendants (a node may be a descendant of itself). BST ordering makes the search simple.

Split-point logic

• If both p and q are smaller than current, move left.

• If both are greater, move right.

• Otherwise current is the split point and therefore the LCA.

```
static TreeNode lowestCommonAncestorBST(
        TreeNode root, int p, int q) {

    TreeNode curr = root;

    while (curr != null) {
        if (p < curr.val && q < curr.val) {
            curr = curr.left;
        } else if (p > curr.val && q > curr.val) {
            curr = curr.right;
        } else {
            return curr;
        }
    }
    return null;
}
```

Example: in the sample tree, LCA(1, 7)=3; LCA(4, 13)=8; LCA(6, 7)=6 because 6 is an ancestor of 7.

Do the keys have to exist?

The code above returns the split node even if one or both keys are absent. Some interview statements guarantee both nodes exist. If not guaranteed and the expected answer must be null when either is absent, first verify both keys with search (two O(h) searches), then compute LCA.

```
static boolean contains(TreeNode root, int key) {
    return searchIterative(root, key) != null;
}

static TreeNode lcaOnlyIfBothExist(TreeNode root, int p, int q) {
    if (!contains(root, p) || !contains(root, q)) return null;
    return lowestCommonAncestorBST(root, p, q);
}
```

Time: O(h); extra space: O(1).

Important distinction: LCA in a general binary tree is a different problem; it cannot use the BST split-point property unless the input is known to be a BST.

11. Sorted Array to Height-Balanced BST

A sorted array can be converted into a height-balanced BST by repeatedly choosing the middle element as the root. Values to the left form the left subtree; values to the right form the right subtree.

Java solution

```
static TreeNode sortedArrayToBST(int[] nums) {
    return build(nums, 0, nums.length - 1);
}

static TreeNode build(int[] nums, int left, int right) {
    if (left > right) return null;

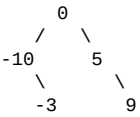
    int mid = left + (right - left) / 2;
    TreeNode node = new TreeNode(nums[mid]);

    node.left = build(nums, left, mid - 1);
    node.right = build(nums, mid + 1, right);

    return node;
}
```

Dry run: [-10, -3, 0, 5, 9]

Middle index is 2, so root = 0. Left range [-10, -3] produces root -10 or -3 depending on the chosen middle convention; right range [5, 9] produces root 5. Every recursive range is halved, so the resulting height is O(log n).




```
This is one valid balanced shape (lower-middle selection on even ranges).
```

Time: O(n), because each array element becomes one node. Auxiliary recursion space: O(log n) for a balanced result, plus O(n) for the created nodes.

Why not insert the sorted array one by one?

Inserting sorted values into an ordinary BST creates a skewed chain: 1 → 2 → 3 → ... . The middle-first construction deliberately chooses a balanced shape.

Precondition: the input must be sorted. For duplicates, decide a consistent duplicate policy; the strict unique-key BST used in this book assumes distinct values.

12. Recover BST — two nodes were swapped

Problem: a valid BST had exactly two node values swapped by mistake. Restore the BST by swapping those two values back. The intended solution is O(n) time and O(h) auxiliary space, often O(1) extra space with Morris traversal.

Key observation

Inorder traversal of a valid BST is strictly increasing. Swapping two values creates one or two inversions (adjacent inorder values where previous > current).

• First inversion: record `first = previous` and `middle = current`.

• If a second inversion occurs: record `last = current`.

• If two swapped nodes were adjacent in inorder, only one inversion occurs; swap first and middle.

• If they were non-adjacent, two inversions occur; swap first and last.

Iterative inorder implementation

```
static void recoverTree(TreeNode root) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    TreeNode prev = null;
    TreeNode first = null;
    TreeNode middle = null;
    TreeNode last = null;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        curr = stack.pop();

        if (prev != null && prev.val > curr.val) {
            if (first == null) {
                first = prev;
                middle = curr;
            } else {
                last = curr;
            }
        }

        prev = curr;
        curr = curr.right;
    }

    if (first != null) {
        if (last != null) {
            int temp = first.val;
            first.val = last.val;
            last.val = temp;
        } else {
            int temp = first.val;
            first.val = middle.val;
            middle.val = temp;
        }
    }
}
```

The method changes node values, not links. It assumes exactly two values were swapped and all other BST structure is intact. If no inversion is found, the tree is already valid under this assumption.

12A. Recover BST: examples and Morris traversal

Adjacent swapped values

Correct inorder: 1, 2, 3, 4, 5. Suppose 2 and 3 are swapped: 1, 3, 2, 4, 5. There is one inversion (3 > 2). Record first=3, middle=2; swap them.

Non-adjacent swapped values

Correct inorder: 1, 2, 3, 4, 5, 6. Suppose 2 and 5 are swapped: 1, 5, 3, 4, 2, 6. Inversions are 5 > 3 and 4 > 2. Record first=5 and last=2; swap them.

Morris traversal idea

Morris inorder temporarily creates a thread from the inorder predecessor (rightmost node of the left subtree) back to the current node. This avoids an explicit stack and recursion, using O(1) auxiliary space. Every temporary thread must be removed before moving on.

```
static void recoverTreeMorris(TreeNode root) {
    TreeNode curr = root;
    TreeNode prev = null, first = null, middle = null, last = null;

    while (curr != null) {
        if (curr.left == null) {
            // Visit curr.
            if (prev != null && prev.val > curr.val) {
                if (first == null) {
                    first = prev;
                    middle = curr;
                } else {
                    last = curr;
                }
            }
            prev = curr;
            curr = curr.right;
        } else {
            TreeNode pred = curr.left;
            while (pred.right != null && pred.right != curr) {
                pred = pred.right;
            }

            if (pred.right == null) {
                pred.right = curr;       // create temporary thread
                curr = curr.left;
            } else {
                pred.right = null;       // remove thread
                // Visit curr.
                if (prev != null && prev.val > curr.val) {
                    if (first == null) {
                        first = prev;
                        middle = curr;
                    } else {
                        last = curr;
                    }
                }
                prev = curr;
                curr = curr.right;
            }
        }
    }

    if (first != null) {
        TreeNode second = (last != null) ? last : middle;
        int temp = first.val;
        first.val = second.val;
        second.val = temp;
    }
}
```

Morris recovery: O(n) time and O(1) auxiliary space. It temporarily mutates pointers during traversal but restores every thread. Do not return early after detecting an inversion; finish traversal so all temporary links are removed.

Safety rule: Morris traversal must complete its cleanup. An exception or early return while a thread exists can leave the tree corrupted.

13. Full reusable BST class

This consolidated class brings the main operations together. It uses unique keys and ignores duplicates. Paste into a Java file named `BSTDemo.java`.

```
import java.util.*;

public class BSTDemo {
    static class Node {
        int val;
        Node left, right;
        Node(int val) { this.val = val; }
    }

    static class BST {
        Node root;

        Node search(int key) {
            Node curr = root;
            while (curr != null) {
                if (key == curr.val) return curr;
                curr = (key < curr.val) ? curr.left : curr.right;
            }
            return null;
        }

        void insert(int value) {
            root = insert(root, value);
        }

        private Node insert(Node node, int value) {
            if (node == null) return new Node(value);
            if (value < node.val) node.left = insert(node.left, value);
            else if (value > node.val) node.right = insert(node.right, value);
            return node;
        }

        Node min(Node node) {
            if (node == null) return null;
            while (node.left != null) node = node.left;
            return node;
        }

        Node max(Node node) {
            if (node == null) return null;
            while (node.right != null) node = node.right;
            return node;
        }

        void delete(int key) {
            root = delete(root, key);
        }

        private Node delete(Node node, int key) {
            if (node == null) return null;
            if (key < node.val) node.left = delete(node.left, key);
            else if (key > node.val) node.right = delete(node.right, key);
            else {
                if (node.left == null) return node.right;
                if (node.right == null) return node.left;
                Node s = min(node.right);
                node.val = s.val;
                node.right = delete(node.right, s.val);
            }
            return node;
        }

        boolean contains(int key) {
            return search(key) != null;
        }

        void printInorder() {
            printInorder(root);
            System.out.println();
        }

        private void printInorder(Node node) {
            if (node == null) return;
            printInorder(node.left);
            System.out.print(node.val + " ");
            printInorder(node.right);
        }
```

```
    }

    public static void main(String[] args) {
        BST tree = new BST();
        int[] a = {8, 3, 10, 1, 6, 14, 4, 7, 13};

        for (int x : a) tree.insert(x);
        tree.printInorder();

        System.out.println(tree.contains(7));  // true
        System.out.println(tree.min(tree.root).val); // 1
        System.out.println(tree.max(tree.root).val); // 14

        tree.delete(3);
        tree.printInorder();
    }
}
```

Expected inorder before deletion: `1 3 4 6 7 8 10 13 14`. After deleting 3: `1 4 6 7 8 10 13 14`.

14. Complexity master sheet

Operation	Time	Extra space	Notes
Search	O(h)	O(1) iterative / O(h) recursive	One root-to-leaf path
Insert	O(h)	O(1) iterative / O(h) recursive	Duplicates policy matters
Delete	O(h)	O(h) recursive / O(1) iterative	Successor search included
Minimum / Maximum	O(h)	O(1)	Walk leftmost / rightmost
Validate BST	O(n)	O(h)	Bounds or inorder
Kth smallest/largest	O(h + k)	O(h)	Can stop early
Successor / predecessor	O(h)	O(1)	Candidate tracking
LCA in BST	O(h)	O(1)	Split point
Sorted array → BST	O(n)	O(log n) recursion	Creates n nodes
Recover BST (stack)	O(n)	O(h)	Inorder inversions
Recover BST (Morris)	O(n)	O(1)	Must restore threads

h is tree height. Balanced BST: h = O(log n). Skewed BST: h = O(n).

Do not say every BST operation is O(log n)

A plain BST does not automatically balance itself. Sorted insertion can produce height n-1. The safe interview answer is O(h), with balanced and worst-case bounds.

15. Edge cases checklist

• Empty tree: search returns null; min/max return null; delete leaves it empty; validation returns true.

• Single node: deleting its key must set root to null.

• Deleting root with only a left child: new root becomes left child.

• Deleting root with only a right child: new root becomes right child.

• Deleting a node with two children: successor must be removed from its original position.

• Deleting a missing key: tree must remain unchanged.

• Inserting duplicate key: follow one documented policy; this book ignores duplicates.

• Validate with Integer.MIN_VALUE and Integer.MAX_VALUE: use long bounds.

• Kth query with k <= 0 or k > n: return a documented no-answer result.

• Successor of maximum and predecessor of minimum: no answer.

• LCA when one key is absent: clarify whether the problem guarantees both exist.

• Sorted array input: verify it is actually sorted and decide duplicate handling.

• Recover BST: assume exactly two values are swapped; do not confuse it with arbitrary corruption.

A small test matrix

Test	Input / action	Expected check
Empty	search(5), min, max	null, null, null
Root removal	insert 5; delete 5	root == null
Leaf removal	insert 5,3,7; delete 3	inorder [5,7]
One child	insert 5,3, delete 5	root is 3
Two children	insert 5,3,7,6; delete 5	inorder [3,6,7]
Missing delete	delete 99	unchanged inorder
Boundary values	insert MIN, 0, MAX	validation true
Invalid deep node	10,5,15,6 under 15.left	validation false

16. Interview questions and variations

Core questions

• Why is inorder traversal sorted in a BST?

• Why is checking only immediate children not enough to validate a BST?

• What are the three deletion cases? Why does successor deletion become simpler?

• Why can a BST become O(n) even though search is often described as O(log n)?

• What is the difference between successor and right child?

• How do you find kth smallest without storing the whole inorder list?

• How would you handle duplicate keys?

• How do you validate a BST containing integer boundary values?

• Why does sorted-array construction choose the middle element?

• How can two swapped BST nodes be detected from inorder traversal?

Practice set — solve in this order

Level	Problem	Main idea
1	Search in BST	One-path comparison
1	Insert into BST	Find null child position
1	Minimum / Maximum	Extreme left / right
1	Delete a node	Three structural cases
2	Validate BST	Global bounds
2	Kth smallest	Inorder with counter
2	Kth largest	Reverse inorder
2	Successor / predecessor	Subtree extreme + ancestor candidate
2	LCA in BST	Split point
2	Sorted array to BST	Middle recursion
3	Recover BST	Inorder inversions
3	Recover BST with O(1) space	Morris traversal
3	Floor and ceiling in BST	Track best candidate
3	Range sum in BST	Prune irrelevant branches
3	Trim BST to [low, high]	Discard out-of-range roots
3	Two Sum in BST	Inorder list or two iterators
3	Convert BST to greater-sum tree	Reverse inorder running sum
3	Balance an existing BST	Inorder array + rebuild

17. Bonus interview patterns

Floor and ceiling

Floor(x) is the greatest key ≤ x. Ceiling(x) is the smallest key ≥ x. Search like normal BST search while remembering the best candidate whenever moving in a direction that may contain the answer.

```
static Integer floor(TreeNode root, int x) {
    Integer ans = null;
    while (root != null) {
        if (root.val == x) return x;
        if (root.val < x) {
            ans = root.val;
            root = root.right;
        } else {
            root = root.left;
        }
    }
    return ans;
}

static Integer ceiling(TreeNode root, int x) {
    Integer ans = null;
    while (root != null) {
        if (root.val == x) return x;
        if (root.val > x) {
            ans = root.val;
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return ans;
}
```

Range sum

For sum of values in [low, high], skip the left subtree if root.val < low; skip the right subtree if root.val > high.

```
static int rangeSum(TreeNode root, int low, int high) {
    if (root == null) return 0;

    if (root.val < low) {
        return rangeSum(root.right, low, high);
    }
    if (root.val > high) {
        return rangeSum(root.left, low, high);
    }

    return root.val
        + rangeSum(root.left, low, high)
        + rangeSum(root.right, low, high);
}
```

Worst-case time O(n), but pruning can make it much faster. If the range covers nearly all keys, most nodes must be visited.

Trim BST

```
static TreeNode trimBST(TreeNode root, int low, int high) {
    if (root == null) return null;

    if (root.val < low) {
        return trimBST(root.right, low, high);
    }
    if (root.val > high) {
        return trimBST(root.left, low, high);
    }

    root.left = trimBST(root.left, low, high);
    root.right = trimBST(root.right, low, high);
    return root;
}
```

The retained tree remains a BST. Values outside the interval are removed; relative ordering of retained nodes is unchanged.

18. Final revision sheet

Concept	Remember this
BST rule	All left-subtree keys < node < all right-subtree keys
Search	Compare; discard one side
Insert	Attach at first null position
Delete leaf	Replace with null
Delete one child	Replace node with its child
Delete two children	Copy inorder successor; delete successor
Minimum	Leftmost node
Maximum	Rightmost node
Validate	Global lower/upper bounds
Kth smallest	Inorder; stop at k
Kth largest	Reverse inorder; stop at k
Successor	Next greater; right-subtree minimum or ancestor candidate
Predecessor	Next smaller; left-subtree maximum or ancestor candidate
LCA BST	First split point
Sorted array	Middle becomes root recursively
Recover BST	Detect inorder inversions and swap values

Explain delete in 30 seconds

“I first search for the key. If it is a leaf, I remove it. If it has one child, I connect its parent directly to that child. If it has two children, I find the smallest node in its right subtree, copy that successor value into the target, and delete the successor from its old position. The operation is O(h) time.”

Explain validation in 20 seconds

“I pass a valid open interval down the recursion. A node must be greater than its lower bound and less than its upper bound. Going left tightens the upper bound; going right tightens the lower bound. I use long bounds so integer extremes are valid.”

Final self-check

• Can you code search and insert without looking?

• Can you explain why delete with two children uses successor or predecessor?

• Can you validate a deep subtree violation?

• Can you get kth smallest and largest without sorting?

• Can you find successor even when there is no right subtree?

• Can you explain why a plain BST can degrade to O(n)?

• Can you recover both adjacent and non-adjacent swapped nodes?

If you can answer these and implement the listed methods from memory, you have a strong foundation for BST interview questions.